

TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based ETL pipeline. It defines the technical architecture, data structures, transformation logic, and runtime parameters required to build a production-grade solution that ingests customer and order data, implements data quality operations, applies SCD Type 2 logic using Apache Hudi, and generates analytical aggregations.

Scope

This TRD provides technical specifications for:

- AWS Glue job configuration and runtime parameters
- S3 data ingestion patterns for CSV files
- Data quality and deduplication algorithms
- AWS Glue Data Catalog table registration and metadata management
- Apache Hudi table implementation with SCD Type 2 logic
- Incremental processing patterns for change data capture
- Aggregation logic and analytics table generation
- Error handling, logging, and operational monitoring

This TRD does NOT cover:

- Infrastructure provisioning (VPC, subnets, security groups)
- IAM role and policy definitions
- Cost optimization strategies
- Disaster recovery procedures
- Data retention and archival automation

Assumptions

1. AWS Glue version 3.0 or higher with Spark 3.1+ and Python 3.7+ support
2. Apache Hudi libraries (version 0.10.0 or higher) are available in the Glue job environment
3. CSV files use comma (`,`) as delimiter and double-quote (`"`) as text qualifier
4. CSV files include header rows with column names matching the specified schema

5. Date fields in Order data are in ISO 8601 format (YYYY-MM-DD) or standard parseable format
6. Numeric fields (PricePerUnit, Qty) are stored as strings in CSV and require casting to decimal/integer
7. CustId and OrderId are string types suitable for use as primary/foreign keys
8. The string "Null" (case-sensitive) and actual NULL values both represent missing data
9. Duplicate detection uses exact string matching across all columns
10. SCD2 tracking fields (IsActive, StartDate, EndDate, OpTs) use timestamp data type
11. Hudi precombine field is OpTs; record key is a composite of CustId and OrderId
12. AWS Glue job has IAM permissions for S3 read/write, Glue Catalog access, and CloudWatch logging
13. S3 bucket `adif-sdlc` exists and is accessible in the same AWS region as the Glue job
14. Glue database `gen\_ai\_poc\_databrickscoe` will be created if it does not exist
15. Incremental processing uses a watermark mechanism to track last processed timestamp
16. Aggregation calculations assume PricePerUnit and Qty are non-negative numeric values
17. Glue job runs in a VPC with appropriate network connectivity to S3 endpoints
18. CloudWatch log retention is configured at the AWS account level
19. Hudi table compaction and cleaning are handled by separate maintenance jobs (not in scope)
20. Data type inference from CSV files follows standard Spark CSV reader behavior

•

2. Technical Requirements

TR-INGEST-001: Implement Customer Data Ingestion from S3

- Technical Requirement ID: TR-INGEST-001
- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 using AWS Glue DynamicFrame or Spark DataFrame, parse CSV content, and load into memory for downstream processing.

- Source Details:
- Source Type: AWS S3
- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/
- Source Format: CSV
- Source File Pattern: .csv
- Delimiter: `,` (comma)
- Header: Present (first row contains column names)

- Text Qualifier: `` (double quote)
- Target Details:
- Target Type: In-memory DataFrame/DynamicFrame
- Target Name: customer_raw_df
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	Yes	
	EmailId	string	-	Yes	
	Region	string	-	Yes	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	Yes	
	EmailId	string	-	Yes	
	Region	string	-	Yes	

- Processing / Transformation Logic:

1. Initialize AWS Glue context and Spark session
2. Use Glue's `create_dynamic_frame.from_options()` or Spark's `spark.read.csv()` to read from S3 path
3. Configure CSV reader options:
 - `format`:"csv"`
 - `header`:"true"`
 - `inferSchema`:"false"` (use explicit schema)
 - `delimiter`:","`
 - `quote`:"\""`
 - `escape`:"\""`
 - `multiLine`:"false"`
4. Apply explicit schema definition for CustId, Name, EmailId, Region as string types
5. Load all CSV files matching the path pattern into a single DataFrame
6. Cache the DataFrame in memory for subsequent operations

- Validation Rules:

- Verify S3 path exists and is accessible
- Verify at least one CSV file is present in the path
- Verify CSV files contain the expected header row with columns: CustId, Name, EmailId, Region
- Verify all rows have exactly 4 columns after parsing
- Error Handling and Logging:
 - If S3 path does not exist: Log error "Source path not found: s3://adif-sdlc/sdlc_wizard/customerdata/" and raise exception
 - If no CSV files found: Log error "No customer data files found in s3://adif-sdlc/sdlc_wizard/customerdata/" and raise exception
 - If CSV parsing fails: Log error "Invalid CSV format in customer data" with file name and line number, raise exception
 - If schema mismatch detected: Log error "Customer CSV schema does not match expected columns" and raise exception
 - Log info message: "Successfully loaded {row_count} customer records from S3"
- Runtime Parameters:
 - `customer\_source\_path`: s3://adif-sdlc/sdlc_wizard/customerdata/
 - `csv\_delimiter`: ","
 - `csv\_header`: "true"
- Operational Notes:
 - Use Glue job bookmark to track processed files if incremental ingestion is required in future iterations
 - Monitor S3 request metrics in CloudWatch for read throughput and latency
 - Consider enabling S3 server access logging for audit trail
-

TR-INGEST-002: Implement Order Data Ingestion from S3

- Technical Requirement ID: TR-INGEST-002
- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order CSV files from S3 using AWS Glue DynamicFrame or Spark DataFrame, parse CSV content, and load into memory for downstream processing.

- Source Details:
 - Source Type: AWS S3
 - Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/

- Source Format: CSV
- Source File Pattern: .csv
- Delimiter: `,` (comma)
- Header: Present (first row contains column names)
- Text Qualifier: `"` (double quote)
- Target Details:
- Target Type: In-memory DataFrame/DynamicFrame
- Target Name: order_raw_df

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	Yes	
	PricePerUnit	string	-	Yes	
	Qty	string	-	Yes	
	Date	string	-	Yes	
	CustId	string	-	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	Yes	
	PricePerUnit	string	-	Yes	
	Qty	string	-	Yes	
	Date	string	-	Yes	
	CustId	string	-	No	

- Processing / Transformation Logic:

1. Initialize AWS Glue context and Spark session
2. Use Glue's `create\_dynamic\_frame.from\_options()` or Spark's `spark.read.csv()` to read from S3 path
3. Configure CSV reader options:
 - `format`: "csv"
 - `header`: "true"
 - `inferSchema`: "false" (use explicit schema)
 - `delimiter`: ","

- `quote`: "\""
 - `escape`: "\\"
 - `multiLine`: "false"
4. Apply explicit schema definition for OrderId, ItemName, PricePerUnit, Qty, Date, CustId as string types
 5. Load all CSV files matching the path pattern into a single DataFrame
 6. Cache the DataFrame in memory for subsequent operations
- Validation Rules:
 - Verify S3 path exists and is accessible
 - Verify at least one CSV file is present in the path
 - Verify CSV files contain the expected header row with columns: OrderId, ItemName, PricePerUnit, Qty, Date, CustId
 - Verify all rows have exactly 6 columns after parsing
 - Error Handling and Logging:
 - If S3 path does not exist: Log error "Source path not found: s3://adif-sdlc/sdlc_wizard/orderdata/" and raise exception
 - If no CSV files found: Log error "No order data files found in s3://adif-sdlc/sdlc_wizard/orderdata/" and raise exception
 - If CSV parsing fails: Log error "Invalid CSV format in order data" with file name and line number, raise exception
 - If schema mismatch detected: Log error "Order CSV schema does not match expected columns" and raise exception
 - Log info message: "Successfully loaded {row_count} order records from S3"
 - Runtime Parameters:
 - `order\_source\_path`: s3://adif-sdlc/sdlc_wizard/orderdata/
 - `csv\_delimiter`: ","
 - `csv\_header`: "true"
 - Operational Notes:
 - Use Glue job bookmark to track processed files if incremental ingestion is required in future iterations
 - Monitor S3 request metrics in CloudWatch for read throughput and latency
 - Consider enabling S3 server access logging for audit trail
 -

TR-CLEAN-001: Implement Data Quality Operations for Customer Data

- Technical Requirement ID: TR-CLEAN-001
- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the customer dataset using Spark DataFrame operations.

- Source Details:
- Source Type: In-memory DataFrame
- Source Name: customer_raw_df (output from TR-INGEST-001)
- Target Details:
- Target Type: In-memory DataFrame
- Target Name: customer_clean_df

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	Yes	
	EmailId	string	-	Yes	
	Region	string	-	Yes	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Processing / Transformation Logic:

1. Count initial record count: ``initial_count = customer_raw_df.count()``
2. Remove rows where any column contains the string "Null" (case-sensitive):
 - Apply filter: ``customer_raw_df.filter(~(col("CustId") == "Null") & ~(col("Name") == "Null") & ~(col("EmailId") == "Null") & ~(col("Region") == "Null"))``
3. Remove rows where any column contains actual NULL values:
 - Apply filter: ``customer_raw_df.filter(col("CustId").isNotNull() & col("Name").isNotNull() & col("EmailId").isNotNull() & col("Region").isNotNull())``
4. Count records after null removal: ``null_removed_count = customer_raw_df.count()``

5. Remove duplicate rows based on all columns:

- Apply deduplication: ``customer_raw_df.dropDuplicates(["CustId", "Name", "EmailId", "Region"])`

- Retain first occurrence of each duplicate set (default Spark behavior)

6. Count final record count: ``final_count = customer_clean_df.count()`

7. Calculate metrics:

- ``null_removed = initial_count - null_removed_count``

- ``duplicates_removed = null_removed_count - final_count``

8. Cache cleaned DataFrame for downstream operations

- Validation Rules:

- Verify final dataset contains at least one record

- Verify no NULL values exist in any column after cleaning

- Verify no string "Null" values exist in any column after cleaning

- Verify no duplicate rows exist after deduplication

- Error Handling and Logging:

- If all records removed due to nulls: Log warning "All customer records contained null values" and continue with empty DataFrame

- Log info message: "Customer data cleaning: {null_removed} records removed due to nulls, {duplicates_removed} duplicates removed"

- Log info message: "Customer data cleaning complete: {final_count} valid records remaining"

- If final count is zero: Log warning "No valid customer records after cleaning"

- Runtime Parameters:

- ``null_string_value``: "Null"

- Operational Notes:

- Deduplication uses Spark's default hash-based algorithm

- Consider using ``coalesce()` after deduplication to reduce partition count if dataset is small

- Monitor Spark UI for shuffle operations during deduplication

-

TR-CLEAN-002: Implement Data Quality Operations for Order Data

- Technical Requirement ID: TR-CLEAN-002

- Related Functional Requirement ID(s): FR-CLEAN-002

- Objective:

Remove records containing null values (both string "Null" and actual NULL) and eliminate duplicate records from the order dataset using Spark DataFrame operations.

- Source Details:
- Source Type: In-memory DataFrame
- Source Name: order_raw_df (output from TR-INGEST-002)

- Target Details:
- Target Type: In-memory DataFrame
- Target Name: order_clean_df

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	Yes	
	PricePerUnit	string	-	Yes	
	Qty	string	-	Yes	
	Date	string	-	Yes	
	CustId	string	-	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

- Processing / Transformation Logic:

1. Count initial record count: ``initial_count = order_raw_df.count()``

2. Remove rows where any column contains the string "Null" (case-sensitive):

• Apply filter: ``order_raw_df.filter(~(col("OrderId") == "Null") & ~(col("ItemName") == "Null") & ~(col("PricePerUnit") == "Null") & ~(col("Qty") == "Null") & ~(col("Date") == "Null") & ~(col("CustId") == "Null"))``

3. Remove rows where any column contains actual NULL values:

• Apply filter: ``order_raw_df.filter(col("OrderId").isNotNull() & col("ItemName").isNotNull() & col("PricePerUnit").isNotNull() & col("Qty").isNotNull() & col("Date").isNotNull() & col("CustId").isNotNull())``

4. Count records after null removal: ``null_removed_count = order_raw_df.count()``
5. Remove duplicate rows based on all columns:
 - Apply deduplication: ``order_raw_df.dropDuplicates(["OrderId", "ItemName", "PricePerUnit", "Qty", "Date", "CustId"])``
 - Retain first occurrence of each duplicate set (default Spark behavior)
6. Count final record count: ``final_count = order_clean_df.count()``
7. Calculate metrics:
 - ``null_removed = initial_count - null_removed_count``
 - ``duplicates_removed = null_removed_count - final_count``
8. Cache cleaned DataFrame for downstream operations
 - Validation Rules:
 - Verify final dataset contains at least one record
 - Verify no NULL values exist in any column after cleaning
 - Verify no string "Null" values exist in any column after cleaning
 - Verify no duplicate rows exist after deduplication
 - Error Handling and Logging:
 - If all records removed due to nulls: Log warning "All order records contained null values" and continue with empty DataFrame
 - Log info message: "Order data cleaning: {null_removed} records removed due to nulls, {duplicates_removed} duplicates removed"
 - Log info message: "Order data cleaning complete: {final_count} valid records remaining"
 - If final count is zero: Log warning "No valid order records after cleaning"
 - Runtime Parameters:
 - ``null_string_value``: "Null"
 - Operational Notes:
 - Deduplication uses Spark's default hash-based algorithm
 - Consider using ``coalesce()`` after deduplication to reduce partition count if dataset is small
 - Monitor Spark UI for shuffle operations during deduplication
 -

TR-CATALOG-001: Register Customer Table in AWS Glue Data Catalog

- Technical Requirement ID: TR-CATALOG-001
- Related Functional Requirement ID(s): FR-CATALOG-001

- Objective:

Register the cleaned customer dataset as a table in AWS Glue Data Catalog to enable SQL querying via Amazon Athena and maintain metadata for data governance.

- Source Details:

- Source Type: In-memory DataFrame
- Source Name: customer_clean_df (output from TR-CLEAN-001)

- Target Details:

- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_customer
- Storage Location: Not specified in FRD (table registration only, no physical write)

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Processing / Transformation Logic:

1. Initialize AWS Glue client using boto3
2. Check if database "gen_ai_poc_databrickscoe" exists:
 - Use ``glue_client.get_database(Name='gen_ai_poc_databrickscoe')``
 - If database does not exist, create it: ``glue_client.create_database(DatabaseInput={'Name': 'gen_ai_poc_databrickscoe'})``
3. Define table schema with columns: CustId (string), Name (string), EmailId (string), Region (string)
4. Create or update table "sdlc_wizard_customer":
 - Use ``glue_client.create_table()`` if table does not exist

- Use ``glue_client.update_table()`` if table exists
5. Set table properties:
 - ``classification``: "csv" or appropriate format
 - ``compressionType``: "none" or as applicable
 6. Register table metadata including column names, data types, and storage location (if applicable)
 7. Verify table registration by querying Glue Catalog
 - Validation Rules:
 - Verify database "gen_ai_poc_databrickscoe" exists or is successfully created
 - Verify table "sdhc_wizard_customer" is registered in Glue Catalog
 - Verify table schema matches expected columns and data types
 - Error Handling and Logging:
 - If database creation fails: Log error "Unable to create Glue database: gen_ai_poc_databrickscoe" and raise exception
 - If table registration fails: Log error "Unable to register customer table in Glue Catalog" and raise exception
 - Log info message: "Successfully registered table gen_ai_poc_databrickscoe.sdhc_wizard_customer in Glue Catalog"
 - Runtime Parameters:
 - ``glue_database``: gen_ai_poc_databrickscoe
 - ``glue_table_customer``: sdhc_wizard_customer
 - Operational Notes:
 - Table registration does not write data to S3; it only creates metadata in Glue Catalog
 - If physical storage is required, use Glue's ``write_dynamic_frame.from_options()`` to write data to S3 before catalog registration
 - Ensure IAM role has ``glue:CreateDatabase``, ``glue:CreateTable``, and ``glue:UpdateTable`` permissions

TR-CATALOG-002: Register Order Table in AWS Glue Data Catalog

- Technical Requirement ID: TR-CATALOG-002
- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Register the cleaned order dataset as a table in AWS Glue Data Catalog to enable SQL querying via Amazon Athena and maintain metadata for data governance.

- Source Details:
- Source Type: In-memory DataFrame
- Source Name: order_clean_df (output from TR-CLEAN-002)
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_order
- Storage Location: Not specified in FRD (table registration only, no physical write)
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

- Processing / Transformation Logic:
 1. Initialize AWS Glue client using boto3
 2. Check if database "gen_ai_poc_databrickscoe" exists:
 - Use ``glue_client.get_database(Name='gen_ai_poc_databrickscoe')``
 - If database does not exist, create it: ``glue_client.create_database(DatabaseInput={'Name': 'gen_ai_poc_databrickscoe'})``
 3. Define table schema with columns: OrderId (string), ItemName (string), PricePerUnit (string), Qty (string), Date (string), CustId (string)
 4. Create or update table "sdlc_wizard_order":

- Use ``glue_client.create_table()`` if table does not exist
 - Use ``glue_client.update_table()`` if table exists
5. Set table properties:
- ``classification``: "csv" or appropriate format
 - ``compressionType``: "none" or as applicable
6. Register table metadata including column names, data types, and storage location (if applicable)
7. Verify table registration by querying Glue Catalog
- Validation Rules:
 - Verify database "gen_ai_poc_databrickscoe" exists or is successfully created
 - Verify table "sdlc_wizard_order" is registered in Glue Catalog
 - Verify table schema matches expected columns and data types
 - Error Handling and Logging:
 - If database creation fails: Log error "Unable to create Glue database: gen_ai_poc_databrickscoe" and raise exception
 - If table registration fails: Log error "Unable to register order table in Glue Catalog" and raise exception
 - Log info message: "Successfully registered table gen_ai_poc_databrickscoe.sdlc_wizard_order in Glue Catalog"
 - Runtime Parameters:
 - ``glue_database``: gen_ai_poc_databrickscoe
 - ``glue_table_order``: sdlc_wizard_order
 - Operational Notes:
 - Table registration does not write data to S3; it only creates metadata in Glue Catalog
 - If physical storage is required, use Glue's ``write_dynamic_frame.from_options()`` to write data to S3 before catalog registration
 - Ensure IAM role has ``glue:CreateDatabase``, ``glue:CreateTable``, and ``glue:UpdateTable`` permissions

TR-SCD2-001: Implement SCD Type 2 Logic for Order Summary Using Apache Hudi

- Technical Requirement ID: TR-SCD2-001
- Related Functional Requirement ID(s): FR-SCD2-001
- Objective:

Join customer and order data, implement Slowly Changing Dimension Type 2 logic to track historical changes, and store the result in Apache Hudi format on S3 with registration in Glue Data Catalog.

- Source Details:
- Source Type: In-memory DataFrames
- Source Names:
- customer_clean_df (output from TR-CLEAN-001)
- order_clean_df (output from TR-CLEAN-002)
- Target Details:
- Target Type: Apache Hudi Table on S3
- Target Database: gen_ai_poc_databricksco
- Target Table: ordersummary
- Storage Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Hudi Table Type: Copy-on-Write (COW)
- Hudi Operation: Upsert
- Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

- Output Schema (Order Summary with SCD2 Tracking):

Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----	-----
CustId	string	-	No	Direct map from customer_clean_df.CustId

Name	string	-	No	Direct map from customer_clean_df.Name
EmailId	string	-	No	Direct map from customer_clean_df.EmailId
Region	string	-	No	Direct map from customer_clean_df.Region
OrderId	string	-	No	Direct map from order_clean_df.OrderId
ItemName	string	-	No	Direct map from order_clean_df.ItemName
PricePerUnit	decimal	10,2	No	Cast from order_clean_df.PricePerUnit (string to decimal)
Qty	integer	-	No	Cast from order_clean_df.Qty (string to integer)
Date	date	-	No	Cast from order_clean_df.Date (string to date)
IsActive	boolean	-	No	Derived: true for current records, false for historical records
StartDate	timestamp	-	No	Derived: current timestamp when record becomes active
EndDate	timestamp	-	Yes	Derived: current timestamp when record becomes inactive, null for active record
OpTs	timestamp	-	No	Derived: operation timestamp for Hudi precombine field

- Processing / Transformation Logic:

- Initial Load (First Run):

1. Perform inner join between customer_clean_df and order_clean_df on CustId:

```
...
```

```
joined_df = customer_clean_df.join(order_clean_df, "CustId", "inner")
```

```
...
```

2. Add SCD2 tracking columns:

- `IsActive = true`
- `StartDate = current\_timestamp()`
- `EndDate = null`
- `OpTs = current\_timestamp()`

3. Cast data types:

- `PricePerUnit`: string → decimal(10,2)
- `Qty`: string → integer
- `Date`: string → date (using `to\_date()` function)

4. Configure Hudi write options:

- `hoodie.table.name`: ordersummary
- `hoodie.datasource.write.recordkey.field`: CustId,OrderId (composite key)
- `hoodie.datasource.write.precombine.field`: OpTs
- `hoodie.datasource.write.operation`: upsert
- `hoodie.datasource.write.table.type`: COPY_ON_WRITE
- `hoodie.datasource.hive\_sync.enable`: true
- `hoodie.datasource.hive\_sync.database`: gen_ai_poc_databrickscoe

- `hoodie.datasource.hive\_sync.table`: ordersummary
- `hoodie.datasource.hive\_sync.partition\_extractor\_class`: org.apache.hudi.hive.NonPartitionedExtractor

5. Write DataFrame to S3 using Hudi format:

...

```
joined_df.write.format("hudi").options(hudi_options).mode("overwrite").save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

...

- Incremental Load (Subsequent Runs):

1. Read existing Hudi table from S3:

...

```
existing_hudi_df = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

...

2. Filter active records: `active\_records\_df = existing\_hudi\_df.filter(col("IsActive") == true)`

3. Perform inner join between customer_clean_df and order_clean_df on CustId (new data)

4. For each new record, compare with active records:

- New Record (no match in active records):

- Insert new record with IsActive = true, StartDate = current_timestamp(), EndDate = null, OpTs = current_timestamp()

- Changed Record (match found, customer attributes changed):

- Identify changes in Name, EmailId, or Region

- Create "close" record: copy existing active record, set IsActive = false, EndDate = current_timestamp(), OpTs = current_timestamp()

- Create "new" record: use new customer attributes, set IsActive = true, StartDate = current_timestamp(), EndDate = null, OpTs = current_timestamp()

- Unchanged Record (match found, no changes):

- No action required (existing active record remains)

5. Combine close records and new records into a single DataFrame

6. Write combined DataFrame to Hudi table using upsert operation:

...

```
combined_df.write.format("hudi").options(hudi_options).mode("append").save("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

...

- SCD2 Change Detection Logic:

```
```python
```

```
Pseudocode for change detection
```

```

for each new_record in new_data:
 active_match = active_records.filter(
 (col("CustId") == new_record.CustId) &
 (col("OrderId") == new_record.OrderId) &
 (col("IsActive") == true)
)

 if active_match.count() == 0:
 # New record
 insert_record(new_record, IsActive=true, StartDate=now, EndDate=null, OpTs=now)
 else:
 # Check for changes in customer attributes
 if (active_match.Name != new_record.Name) or
 (active_match.EmailId != new_record.EmailId) or
 (active_match.Region != new_record.Region):
 # Changed record
 close_record(active_match, IsActive=false, EndDate=now, OpTs=now)
 insert_record(new_record, IsActive=true, StartDate=now, EndDate=null, OpTs=now)
 else:
 # Unchanged record - no action
 pass
 ...

```

- Validation Rules:
  - Verify join produces at least one record
  - Verify all active records have IsActive = true and EndDate = null
  - Verify all closed records have IsActive = false and EndDate is not null
  - Verify OpTs is populated for all records
  - Verify PricePerUnit and Qty are successfully cast to numeric types
  - Verify Date is successfully cast to date type
- Error Handling and Logging:
  - If join produces no results: Log warning "No matching customer-order records found" and continue with empty DataFrame
  - If Hudi write fails: Log error "Unable to write to Hudi table at s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/" and raise exception

- If Glue Catalog sync fails: Log error "Unable to register ordersummary table in Glue Catalog" and raise exception
- If type casting fails: Log error "Invalid numeric data in PricePerUnit or Qty" with record details and skip record
- If date parsing fails: Log error "Invalid date format in Date field" with record details and skip record
- Log info message: "SCD2 processing complete: {new\_count} new records, {changed\_count} changed records, {unchanged\_count} unchanged records"
- Runtime Parameters:
  - `hudi\_table\_name`: ordersummary
  - `hudi\_record\_key`: CustId,OrderId
  - `hudi\_precombine\_field`: OpTs
  - `hudi\_table\_type`: COPY\_ON\_WRITE
  - `hudi\_operation`: upsert
  - `hudi\_target\_path`: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
  - `glue\_database`: gen\_ai\_poc\_databrickscoe
  - `glue\_table`: ordersummary
- Operational Notes:
  - Hudi precombine field (OpTs) determines which record to keep when multiple records have the same record key
  - Hudi record key (CustId,OrderId) uniquely identifies each record in the table
  - Hudi Copy-on-Write table type provides snapshot isolation and faster read performance
  - Hudi Hive sync automatically updates Glue Catalog metadata after each write
  - Monitor Hudi table size and run compaction jobs periodically to optimize storage
  - Use Hudi's time-travel queries to access historical snapshots of the table
- 

### ### TR-INCR-001: Implement Incremental Processing for Customer Updates

- Technical Requirement ID: TR-INCR-001
- Related Functional Requirement ID(s): FR-INCR-001
- Objective:

Detect changes in customer data and trigger incremental updates to the ordersummary Hudi table, maintaining SCD Type 2 history without full table reprocessing.

- Source Details:

- Source Type: In-memory DataFrame and Hudi Table
- Source Names:
  - customer\_clean\_df (output from TR-CLEAN-001) - new/updated customer records
  - ordersummary Hudi table from s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/ - existing records
- Target Details:
  - Target Type: Apache Hudi Table on S3
  - Target Database: gen\_ai\_poc\_databricksco
  - Target Table: ordersummary
  - Storage Location: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
- Input Schema (New Customer Data):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Input Schema (Existing Order Summary):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	decimal	10,2	No	
	Qty	integer	-	No	
	Date	date	-	No	
	IsActive	boolean	-	No	
	StartDate	timestamp	-	No	
	EndDate	timestamp	-	Yes	
	OpTs	timestamp	-	No	

- Output Schema:

Same as TR-SCD2-001 output schema

- Processing / Transformation Logic:

- Change Detection:

1. Read existing ordersummary Hudi table from S3:

...

```
existing_hudi_df = spark.read.format("hudi").load("s3://adif-sdlc/curated/sdlc_wizard/ordersummary/")
```

...

2. Filter active records: `active\_records\_df = existing\_hudi\_df.filter(col("IsActive") == true)`

3. Extract unique customer records from active records:

...

```
existing_customers_df = active_records_df.select("CustId", "Name", "EmailId", "Region").distinct()
```

...

4. Perform left anti join to identify new customers:

...

```
new_customers_df = customer_clean_df.join(existing_customers_df, "CustId", "left_anti")
```

...

5. Perform inner join to identify existing customers:

...

```
existing_customers_check_df = customer_clean_df.join(existing_customers_df, "CustId", "inner")
```

...

6. Identify changed customers by comparing attributes:

...

```
changed_customers_df = existing_customers_check_df.filter(
```

(col("customer_clean_df.Name") != col("existing_customers_df.Name"))	
(col("customer_clean_df.EmailId") != col("existing_customers_df.EmailId"))	

```
(col("customer_clean_df.Region") != col("existing_customers_df.Region"))
```

```
)
```

...

7. Combine new and changed customers: `customers\_to\_process\_df = new\_customers\_df.union(changed\_customers\_df)`

- Incremental SCD2 Update:

1. For each customer in customers\_to\_process\_df:

- Retrieve all active order summary records for that CustId:

...

```
active_orders_for_customer = active_records_df.filter(col("CustId") == customer.CustId)
```

...

- For each active order record:
  - Create "close" record: set IsActive = false, EndDate = current\_timestamp(), OpTs = current\_timestamp()
  - Create "new" record: use updated customer attributes, set IsActive = true, StartDate = current\_timestamp(), EndDate = null, OpTs = current\_timestamp()
2. Combine all close records and new records into a single DataFrame
  3. Perform incremental upsert to Hudi table:

...

```
combined_df.write.format("hudi").options(hudi_options).mode("append").save("s3://adif-sdlc/curated/sd
lc_wizard/ordersummary/")
```

...

4. Update Glue Catalog metadata if schema changes occurred
- Validation Rules:
    - Verify change detection logic correctly identifies new and changed customers
    - Verify all affected active records are properly closed (IsActive = false, EndDate populated)
    - Verify new active records are created with correct customer attributes
    - Verify OpTs is updated for all modified records
  - Error Handling and Logging:
    - If no customer changes detected: Log info "No customer updates to process" and exit gracefully
    - If incremental merge fails: Log error "Incremental SCD2 update failed" and raise exception
    - If Glue Catalog update fails: Log error "Unable to update Glue Catalog metadata" and raise exception
    - Log info message: "Incremental processing complete: {new\_customer\_count} new customers, {changed\_customer\_count} changed customers, {total\_records\_updated} records updated"
  - Runtime Parameters:
    - `hudi\_table\_name`: ordersummary
    - `hudi\_record\_key`: CustId,OrderId
    - `hudi\_precombine\_field`: OpTs
    - `hudi\_operation`: upsert
    - `hudi\_target\_path`: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
    - `glue\_database`: gen\_ai\_poc\_databrickscoe
    - `glue\_table`: ordersummary
  - Operational Notes:
    - Incremental processing significantly reduces processing time compared to full table reprocessing

- Use Glue job bookmarks or a watermark table to track last processed timestamp
- Consider implementing a change data capture (CDC) mechanism if source data supports it
- Monitor Hudi table compaction and cleaning to maintain optimal performance
- 

### ### TR-AGG-001: Generate Customer Aggregate Spend Analytics Table

- Technical Requirement ID: TR-AGG-001
- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Aggregate order data by customer to calculate total spending and store the results in a dedicated analytics table on S3 with registration in Glue Data Catalog.

- Source Details:
- Source Type: In-memory DataFrames
- Source Names:
- customer\_clean\_df (output from TR-CLEAN-001)
- order\_clean\_df (output from TR-CLEAN-002)
- Target Details:
- Target Type: Parquet files on S3
- Target Database: gen\_ai\_poc\_databricksco
- Target Table: customeraggregatespend
- Storage Location: s3://adif-sdlc/analytics/customeraggregatespend/
- Write Mode: Overwrite
- Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	

	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

• Output Schema:

Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
-----	-----	-----	-----	-----	
Name	string	-	No	Direct map from customer_clean_df.Name	
TotalAmount	decimal	15,2	No	Derived: SUM(PricePerUnit * Qty) grouped by CustId	
Date	date	-	No	Derived: MAX(Date) grouped by CustId	

• Processing / Transformation Logic:

1. Perform inner join between customer\_clean\_df and order\_clean\_df on CustId:

```
...
```

```
joined_df = customer_clean_df.join(order_clean_df, "CustId", "inner")
```

```
...
```

2. Cast data types for calculation:

- `PricePerUnit`: string → decimal(10,2)
- `Qty`: string → integer
- `Date`: string → date (using `to\_date()` function)

3. Calculate line-level amount:

```
...
```

```
joined_df = joined_df.withColumn("LineAmount", col("PricePerUnit") col("Qty"))
```

```
...
```

4. Group by CustId and Name, aggregate:

- `TotalAmount = SUM(LineAmount)`
- `Date = MAX(Date)` (most recent transaction date)

```
...
```

```
aggregated_df = joined_df.groupBy("CustId", "Name").agg(
 sum("LineAmount").alias("TotalAmount"),
 max("Date").alias("Date")
)
```

```
...
```

5. Select final columns: Name, TotalAmount, Date

```
...
```



```
final_df = aggregated_df.select("Name", "TotalAmount", "Date")
```

```
'''
```

6. Write DataFrame to S3 in Parquet format:

```
'''
```

```
final_df.write.format("parquet").mode("overwrite").save("s3://adif-sdlc/analytics/customeragggregatespend/")
```

```
'''
```

7. Register or update customeragggregatespend table in Glue Data Catalog:

- Use Glue's `write_dynamic_frame.from_catalog()` or boto3 to register table
- Set table properties for Athena querying
- Validation Rules:
  - Verify join produces at least one record
  - Verify TotalAmount is non-negative for all customers
  - Verify Date is a valid date for all customers
  - Verify aggregation produces one row per unique customer (Name)
  - Verify PricePerUnit and Qty are successfully cast to numeric types
- Error Handling and Logging:
  - If join produces no results: Log error "No customer-order matches found for aggregation" and raise exception
  - If TotalAmount calculation fails due to non-numeric values: Log error "Invalid numeric data in PricePerUnit or Qty" with record details and skip record
  - If date parsing fails: Log error "Invalid date format in Date field" with record details and skip record
  - If S3 write fails: Log error "Unable to write aggregate table to S3 at s3://adif-sdlc/analytics/customeragggregatespend/" and raise exception
  - If Glue Catalog registration fails: Log error "Unable to register customeragggregatespend table in Glue Catalog" and raise exception
  - Log info message: "Customer aggregate spend table generated: {customer\_count} customers, total spend: {total\_spend}"
- Runtime Parameters:
  - `aggregate_target_path``: s3://adif-sdlc/analytics/customeragggregatespend/
  - `aggregate_format``: parquet
  - `aggregate_write_mode``: overwrite
  - `glue_database``: gen\_ai\_poc\_databrickscoe
  - `glue_table_aggregate``: customeragggregatespend

- Operational Notes:
- Aggregation uses Spark's distributed groupBy operation for scalability
- Overwrite mode ensures the analytics table reflects the latest data snapshot
- Consider partitioning by Region or Date if the dataset grows large
- Monitor Spark UI for shuffle operations during aggregation
- Use Parquet format for efficient columnar storage and query performance
- 

### ### TR-CATALOG-003: Register Order Summary Table in AWS Glue Data Catalog

- Technical Requirement ID: TR-CATALOG-003
- Related Functional Requirement ID(s): FR-CATALOG-003
- Objective:

Register the ordersummary Hudi table in AWS Glue Data Catalog to enable SQL querying via Amazon Athena with Hudi time-travel support.

- Source Details:
- Source Type: Apache Hudi Table on S3
- Source Path: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen\_ai\_poc\_databrickscoe
- Target Table: ordersummary
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	decimal	10,2	No	
	Qty	integer	-	No	
	Date	date	-	No	
	IsActive	boolean	-	No	

	StartDate	timestamp	-	No	
	EndDate	timestamp	-	Yes	
	OpTs	timestamp	-	No	

- Output Schema:

Same as input schema

- Processing / Transformation Logic:

1. Hudi write operation automatically syncs table metadata to Glue Catalog when ``hoodie.datasource.hive_sync.enable`` is set to true

2. If manual registration is required:

- Initialize AWS Glue client using boto3
- Verify database "gen\_ai\_poc\_databrickscoe" exists
- Create or update table "ordersummary" with Hudi-specific properties:
- ``table_type``: EXTERNAL\_TABLE
- ``parameters``:
- ``EXTERNAL``: TRUE
- ``spark.sql.sources.provider``: hudi
- ``spark.sql.sources.schema.numParts``: 1
- ``spark.sql.sources.schema.part.0``: (full schema JSON)
- Set storage descriptor with Hudi input/output format classes:
- ``inputFormat``: org.apache.hudi.hadoop.HoodieParquetInputFormat
- ``outputFormat``: org.apache.hadoop.hive ql.io.parquet.MapredParquetOutputFormat
- ``serdeInfo.serializationLib``: org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe
- Register table schema including all columns and SCD2 tracking fields

3. Verify table registration by querying Glue Catalog

- Validation Rules:

- Verify database "gen\_ai\_poc\_databrickscoe" exists
- Verify table "ordersummary" is registered in Glue Catalog
- Verify table schema matches expected columns and data types
- Verify Hudi-specific table properties are set correctly

- Error Handling and Logging:

• If table registration fails: Log error "Unable to register ordersummary table in Glue Catalog" and raise exception

• Log info message: "Successfully registered Hudi table gen\_ai\_poc\_databrickscoe.ordersummary in Glue Catalog"

- Runtime Parameters:
- `glue\_database`: gen\_ai\_poc\_databrickscoe
- `glue\_table`: ordersummary
- `hudi\_table\_path`: s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
- Operational Notes:
- Hudi Hive sync automatically handles catalog registration during write operations
- Athena queries can use Hudi time-travel syntax to query historical snapshots
- Ensure Athena workgroup has Hudi connector enabled for querying Hudi tables
- Monitor Glue Catalog for schema evolution and update table metadata as needed
- 

### ### TR-CATALOG-004: Register Customer Aggregate Spend Table in AWS Glue Data Catalog

- Technical Requirement ID: TR-CATALOG-004
- Related Functional Requirement ID(s): FR-CATALOG-004
- Objective:

Register the customeraggregatespend table in AWS Glue Data Catalog to enable SQL querying via Amazon Athena for business reporting and analytics.

- Source Details:
- Source Type: Parquet files on S3
- Source Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen\_ai\_poc\_databrickscoe
- Target Table: customeraggregatespend
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	Name	string	-	No	
	TotalAmount	decimal	15,2	No	
	Date	date	-	No	

- Output Schema:

Same as input schema

- Processing / Transformation Logic:

1. Initialize AWS Glue client using boto3

2. Verify database "gen\_ai\_poc\_databrickscoe" exists

3. Create or update table "customeraggregatespend":

- Use ``glue_client.create_table()`` if table does not exist

- Use ``glue_client.update_table()`` if table exists

4. Set table properties:

- ``table_type`: EXTERNAL_TABLE`

- ``parameters`:`

- ``EXTERNAL`: TRUE`

- ``classification`: parquet`

- Set storage descriptor:

- ``location`: s3://adif-sdlc/analytics/customeraggregatespend/`

- ``inputFormat`: org.apache.hadoop.hive ql.io.parquet.MapredParquetInputFormat`

- ``outputFormat`: org.apache.hadoop.hive ql.io.parquet.MapredParquetOutputFormat`

- ``serdeInfo.serializationLib`: org.apache.hadoop.hive ql.io.parquet.serde.ParquetHiveSerDe`

5. Register table schema with columns: Name (string), TotalAmount (decimal), Date (date)

6. Verify table registration by querying Glue Catalog

- Validation Rules:

- Verify database "gen\_ai\_poc\_databrickscoe" exists

- Verify table "customeraggregatespend" is registered in Glue Catalog

- Verify table schema matches expected columns and data types

- Verify storage location points to correct S3 path

- Error Handling and Logging:

- If table registration fails: Log error "Unable to register customeraggregatespend table in Glue Catalog" and raise exception

- Log info message: "Successfully registered table gen\_ai\_poc\_databrickscoe.customeraggregatespend in Glue Catalog"

- Runtime Parameters:

- ``glue_database`: gen_ai_poc_databrickscoe`

- ``glue_table_aggregate`: customeraggregatespend`

- ``aggregate_table_path`: s3://adif-sdlc/analytics/customeraggregatespend/`

- Operational Notes:

- Parquet format provides efficient columnar storage for analytics queries

- Athena queries can directly access the table for business intelligence reporting
- Consider adding table descriptions and column comments for data governance
- Monitor query performance in Athena and optimize partitioning if needed
- 

### ## 3. Runtime Strategy Notes

#### ### Authoring Language

- Primary Language: Python 3.7+
- Framework: PySpark 3.1+ (AWS Glue 3.0 or higher)
- Libraries:
- Apache Hudi 0.10.0+ (for SCD2 implementation)
- boto3 (for AWS Glue Catalog and S3 operations)
- pyspark.sql.functions (for data transformations)

#### ### Execution Strategy

- Job Type: AWS Glue ETL Job (Spark)
- Execution Mode: Batch processing
- Orchestration: AWS Glue Workflow or AWS Step Functions (recommended for multi-stage pipeline)
- Scheduling: AWS Glue Triggers or Amazon EventBridge (for scheduled or event-driven execution)
- Parallelism: Leverage Spark's distributed processing with appropriate DPU allocation
- Checkpointing: Use Glue job bookmarks for incremental processing (TR-INCR-001)

#### ### Glue Job Type Expectations

- Glue Version: 3.0 or higher
- Job Type: Spark (Python Shell not suitable due to Hudi and large-scale processing requirements)
- Worker Type: G.1X or G.2X (depending on data volume)
- Number of Workers: Start with 2-5 workers, scale based on data volume and performance requirements
- Timeout: 2880 minutes (48 hours) - adjust based on expected processing time
- Max Retries: 1-2 (for transient failures)
- Job Bookmark: Enabled for incremental processing
- Continuous Logging: Enabled for real-time monitoring in CloudWatch
- Spark UI: Enabled for performance tuning and debugging
-

## ## 4. Data Design Details

### ### Source Paths / Source Systems

- Customer Data Source: s3://adif-sdlc/sdlc\_wizard/customerdata/ (CSV files)
- Order Data Source: s3://adif-sdlc/sdlc\_wizard/orderdata/ (CSV files)
- No intermediate sources or staging tables are introduced

### ### Target Paths / Target Tables

- Curated Layer:
- Order Summary (Hudi): s3://adif-sdlc/curated/sdlc\_wizard/ordersummary/
- Glue Catalog Table: gen\_ai\_poc\_databrickscoe.ordersummary
- Analytics Layer:
- Customer Aggregate Spend (Parquet): s3://adif-sdlc/analytics/customeraggregatespend/
- Glue Catalog Table: gen\_ai\_poc\_databrickscoe.customeraggregatespend
- Catalog-Only Tables (no physical write specified in FRD):
- Customer Table: gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_customer
- Order Table: gen\_ai\_poc\_databrickscoe.sdlc\_wizard\_order

### ### Catalog Registration Expectations

- All tables must be registered in AWS Glue Data Catalog under database `gen\_ai\_poc\_databrickscoe`
- Database will be created if it does not exist
- Table metadata includes column names, data types, storage location, and format
- Hudi tables require Hudi-specific properties for Athena querying
- Parquet tables use standard Parquet SerDe for Athena querying

### ### Partitioning Expectations

- Order Summary (Hudi): No partitioning specified in FRD (non-partitioned table)
- Customer Aggregate Spend (Parquet): No partitioning specified in FRD (non-partitioned table)
- Future Enhancement: Consider partitioning by Region or Date if data volume grows

### ### File Formats / Table Formats

- Source Files: CSV (comma-delimited, with header row)
- Curated Layer: Apache Hudi (Copy-on-Write table type)
- Analytics Layer: Parquet (columnar format)
- Catalog Tables: Metadata only (no physical storage for sdlc\_wizard\_customer and sdlc\_wizard\_order)

### ### Update / Overwrite / Append Behavior

- Order Summary (Hudi):
- Initial Load: Overwrite mode (first run)
- Incremental Load: Upsert mode (subsequent runs)
- SCD2 Logic: Upsert operation closes old records and inserts new active records
- Customer Aggregate Spend (Parquet):
- Write Mode: Overwrite (full refresh on each run)
- Rationale: Analytics table reflects latest snapshot of customer spending

### ### No Staging Tables or Intermediate Layers

- All processing is performed in-memory using Spark DataFrames
- No intermediate staging tables are created in S3 or Glue Catalog
- Source data is read directly from s3://adif-sdlc/sdlc\_wizard/customerdata/ and s3://adif-sdlc/sdlc\_wizard/orderdata/
- Cleaned data is held in memory and used for downstream operations
- Final outputs are written directly to curated and analytics layers
- 

## ## 5. Processing Details

### ### Data Quality Rules

- Null Handling:
- Remove rows where any column contains the string "Null" (case-sensitive)
- Remove rows where any column contains actual NULL values
- Apply to both customer and order datasets before downstream processing
- Deduplication:
- Remove duplicate rows based on all columns in each dataset
- Retain first occurrence of each duplicate set (default Spark behavior)
- Apply to both customer and order datasets after null removal

### ### Deduplication Rules

- Customer Data:
- Deduplication key: CustId, Name, EmailId, Region (all columns)
- Method: Spark's `dropDuplicates()` function
- Retention: First occurrence
- Order Data:
- Deduplication key: OrderId, ItemName, PricePerUnit, Qty, Date, CustId (all columns)
- Method: Spark's `dropDuplicates()` function



- Retention: First occurrence

#### ### Join Rules

- Customer-Order Join (for SCD2 and Aggregation):
  - Join Type: Inner join
  - Join Key: CustId
  - Purpose: Enrich order data with customer attributes
- Result: Only orders with matching customers are included
- Change Detection Join (for Incremental Processing):
  - Join Type: Left anti join (to identify new customers)
  - Join Type: Inner join (to identify existing customers for change detection)
  - Join Key: CustId
  - Purpose: Identify new and changed customer records

#### ### Aggregation Rules

- Customer Aggregate Spend:
  - Group By: CustId, Name
  - Aggregations:
    - `TotalAmount = SUM(PricePerUnit Qty)` - total spending per customer
    - `Date = MAX(Date)` - most recent transaction date per customer
  - Output: One row per unique customer (Name)

#### ### SCD / History Tracking Rules

- SCD Type 2 Implementation:
  - Tracking Fields:
    - `IsActive` (boolean): true for current records, false for historical records
    - `StartDate` (timestamp): when record became active
    - `EndDate` (timestamp): when record became inactive (null for active records)
    - `OpTs` (timestamp): operation timestamp for Hudi precombine field
  - Change Detection:
    - Compare customer attributes: Name, EmailId, Region
    - If any attribute changes, trigger SCD2 update
  - Update Logic:
    - Close existing active record: set IsActive = false, EndDate = current\_timestamp(), OpTs = current\_timestamp()
    - Insert new active record: set IsActive = true, StartDate = current\_timestamp(), EndDate = null, OpTs = current\_timestamp()

- Hudi Configuration:
- Record Key: CustId,OrderId (composite key)
- Precombine Field: OpTs (determines which record to keep for same record key)
- Table Type: Copy-on-Write (snapshot isolation, faster reads)
- Operation: Upsert (merge new records with existing records)

### ### Null Handling

- Pre-Processing:
- Remove rows with string "Null" or actual NULL values before any transformations
- Ensures data quality for downstream operations
- Post-Processing:
- SCD2 EndDate field is nullable (null for active records)
- All other fields in final outputs are non-nullable after cleaning

### ### Type Casting Expectations

- Customer Data:
- All fields remain as string type (no casting required)
- Order Data:
- `PricePerUnit`: string → decimal(10,2) (for calculations)
- `Qty`: string → integer (for calculations)
- `Date`: string → date (for date operations and aggregations)
- SCD2 Tracking Fields:
- `IsActive`: boolean
- `StartDate`: timestamp
- `EndDate`: timestamp (nullable)
- `OpTs`: timestamp
- Aggregation Output:
- `TotalAmount`: decimal(15,2) (result of SUM(PricePerUnit Qty))
- `Date`: date (result of MAX(Date))
- 

## ## 6. Traceability Matrix

Requirement ID(s)	Original Source	Technical Source	Description
-----	-----	-----	-----
INGEST-001	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/	Customer data ingestion from S
INGEST-002	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/	Order data ingestion from S

CLEAN-001	customer_raw_df (from TR-INGEST-001)	customer_raw_df (from TR-INGEST-001)	Data quality operations for
CLEAN-002	order_raw_df (from TR-INGEST-002)	order_raw_df (from TR-INGEST-002)	Data quality operations for
CATALOG-001	customer_clean_df (from TR-CLEAN-001)	customer_clean_df (from TR-CLEAN-001)	Register customer table in C
CATALOG-002	order_clean_df (from TR-CLEAN-002)	order_clean_df (from TR-CLEAN-002)	Register order table in Glue
SCD2-001	customer_clean_df, order_clean_df	customer_clean_df, order_clean_df	Implement SCD Type 2 for
INCR-001	customer_clean_df, ordersummary Hudi table	customer_clean_df, ordersummary Hudi table	Incremental processing for
AGG-001	customer_clean_df, order_clean_df	customer_clean_df, order_clean_df	Generate customer aggregat
CATALOG-003	ordersummary Hudi table	ordersummary Hudi table	Register order summary tab
CATALOG-004	customeraggregatespend Parquet files	customeraggregatespend Parquet files	Register customer aggregat

- Source Fidelity Verification:
- All sources in TRD match exactly with sources specified in FRD
- No intermediate sources, staging tables, or alternative sources have been introduced
- In-memory DataFrames are used for processing, maintaining direct lineage to original S3 sources
- Hudi and Parquet outputs are written directly to target paths without intermediate layers
- 

## ## 7. ETL Mapping Requirements

### ### TR-INGEST-001: Customer Data Ingestion

- Source: s3://adif-sdlc/sdlc\_wizard/customerdata/ (CSV)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	Yes	
	EmailId	string	-	Yes	
	Region	string	-	Yes	

- Target: customer\_raw\_df (In-Memory DataFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	string	-	No	Direct map from source CustId	
	Name	string	-	Yes	Direct map from source Name	
	EmailId	string	-	Yes	Direct map from source EmailId	
	Region	string	-	Yes	Direct map from source Region	

- 

### ### TR-INGEST-002: Order Data Ingestion

- Source: s3://adif-sdlc/sdlc\_wizard/orderdata/ (CSV)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	Yes	
	PricePerUnit	string	-	Yes	
	Qty	string	-	Yes	
	Date	string	-	Yes	
	CustId	string	-	No	

- Target: order\_raw\_df (In-Memory DataFrame)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	string	-	No	Direct map from source OrderId	
	ItemName	string	-	Yes	Direct map from source ItemName	
	PricePerUnit	string	-	Yes	Direct map from source PricePerUnit	
	Qty	string	-	Yes	Direct map from source Qty	
	Date	string	-	Yes	Direct map from source Date	
	CustId	string	-	No	Direct map from source CustId	

•

### ### TR-CLEAN-001: Customer Data Cleaning

- Source: customer\_raw\_df

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	Yes	
	EmailId	string	-	Yes	
	Region	string	-	Yes	

- Target: customer\_clean\_df

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	string	-	No	Direct map from source CustId (after null/duplicate removal)	
	Name	string	-	No	Direct map from source Name (after null/duplicate removal)	
	EmailId	string	-	No	Direct map from source EmailId (after null/duplicate removal)	

	Region	string	-	No	Direct map from source Region (after null/duplicate removal)	
--	--------	--------	---	----	--------------------------------------------------------------	--

•

### TR-CLEAN-002: Order Data Cleaning

- Source: order\_raw\_df

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	Yes	
	PricePerUnit	string	-	Yes	
	Qty	string	-	Yes	
	Date	string	-	Yes	
	CustId	string	-	No	

- Target: order\_clean\_df

	Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	string	-	No	Direct map from source OrderId (after null/duplicate removal)	
	ItemName	string	-	No	Direct map from source ItemName (after null/duplicate removal)	
	PricePerUnit	string	-	No	Direct map from source PricePerUnit (after null/duplicate removal)	
	Qty	string	-	No	Direct map from source Qty (after null/duplicate removal)	
	Date	string	-	No	Direct map from source Date (after null/duplicate removal)	
	CustId	string	-	No	Direct map from source CustId (after null/duplicate removal)	

•

### TR-SCD2-001: Order Summary with SCD Type 2

- Source 1: customer\_clean\_df

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	string	-	No	
	Name	string	-	No	
	EmailId	string	-	No	
	Region	string	-	No	

- Source 2: order\_clean\_df

	Column Name	Data Type	Length/Precision/Scale	Nullable	
--	-------------	-----------	------------------------	----------	--

	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

- Target: ordersummary (Hudi Table on S3)

Column Name	Data Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----	-----
CustId	string	-	No	Direct map from customer_clean_df.CustId
Name	string	-	No	Direct map from customer_clean_df.Name
EmailId	string	-	No	Direct map from customer_clean_df.EmailId
Region	string	-	No	Direct map from customer_clean_df.Region
OrderId	string	-	No	Direct map from order_clean_df.OrderId
ItemName	string	-	No	Direct map from order_clean_df.ItemName
PricePerUnit	decimal	10,2	No	Cast from order_clean_df.PricePerUnit (string → decimal)
Qty	integer	-	No	Cast from order_clean_df.Qty (string → integer)
Date	date	-	No	Cast from order_clean_df.Date (string → date)
IsActive	boolean	-	No	Derived: true for current records, false for historical records
StartDate	timestamp	-	No	Derived: current_timestamp() when record becomes active
EndDate	timestamp	-	Yes	Derived: current_timestamp() when record becomes inactive, null for active records
UpdatedAtTs	timestamp	-	No	Derived: current_timestamp() for Hudi precombine field

•

### ### TR-AGG-001: Customer Aggregate Spend

- Source 1: customer\_clean\_df

Column Name	Data Type	Length/Precision/Scale	Nullable
-----	-----	-----	-----
CustId	string	-	No
Name	string	-	No
EmailId	string	-	No
Region	string	-	No

- Source 2: order\_clean\_df

Column Name	Data Type	Length/Precision/Scale	Nullable
-------------	-----------	------------------------	----------

	-----	-----	-----	-----	
	OrderId	string	-	No	
	ItemName	string	-	No	
	PricePerUnit	string	-	No	
	Qty	string	-	No	
	Date	string	-	No	
	CustId	string	-	No	

- Target: customeraggregatespend (Parquet on S3)

Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----
	-	No	Direct map from customer_clean_df.Name
al	15,2	No	Derived: SUM(CAST(order_clean_df.PricePerUnit AS decimal) * CAST(order_clean_df.Qty AS integ
	-	No	Derived: MAX(CAST(order_clean_df.Date AS date)) grouped by CustId

•

## ## Assumptions

- Data Type Assumptions:
  - Exact data types for CSV source columns are not specified in FRD; string type is assumed for all CSV fields
  - Numeric precision and scale for PricePerUnit and TotalAmount are assumed based on typical financial data requirements
  - Date format in source CSV is assumed to be parseable by Spark's ``to_date()`` function
- Schema Assumptions:
  - Customer and Order CSV files include header rows with column names matching the specified schema
  - No additional columns exist in source CSV files beyond those specified in FRD
  - Column order in CSV files matches the order specified in FRD (though Spark can handle variations)
- Processing Assumptions:
  - Glue job has sufficient memory and compute resources to hold cleaned datasets in memory
  - S3 paths are accessible from the Glue job's VPC and IAM role
  - Hudi libraries are pre-installed in the Glue environment or provided via `--extra-jars` parameter
  - Glue job runs in a single AWS region where all S3 buckets and Glue Catalog reside
- Operational Assumptions:

- Glue job is scheduled to run at appropriate intervals (daily, hourly, etc.) based on business requirements
- Source CSV files are immutable once written to S3 (no in-place updates)
- Incremental processing watermark is maintained externally (e.g., in a DynamoDB table or S3 metadata file)
- Hudi table compaction and cleaning are handled by separate maintenance jobs
- Note on Missing Information:
- Storage location for sdlc\_wizard\_customer and sdlc\_wizard\_order tables is not specified in FRD; these are assumed to be catalog-only tables without physical storage
- Partitioning strategy is not specified in FRD; non-partitioned tables are assumed
- Retention policies and data lifecycle management are not specified in FRD and are out of scope
- 
- End of Technical Requirements Document